

Project: Foraging ottimale

ricerca stocastica di risorse, patch model e strategie di leaving

1. Obiettivi della dispensa

Questa dispensa introduce il problema del foraging ottimale come caso di studio per un corso di metodi computazionali per modelli stocastici.

Gli obiettivi sono sei:

1. formalizzare il problema della ricerca di cibo in un ambiente a patch come processo stocastico;
2. derivare il teorema del valore marginale come soluzione deterministica del problema di ottimizzazione;
3. introdurre varianti stocastiche del modello in cui la qualità delle patch e il tempo di viaggio sono variabili casuali;
4. definire e confrontare diverse strategie di leaving: tempo fisso, soglia di tasso, regola approssimata del valore marginale;
5. simulare traiettorie di foraging e stimare il tasso di intake energetico medio;
6. discutere le connessioni tra foraging ottimale, random walk e processi di rinforzo.

Dal punto di vista del corso, questo modello è particolarmente interessante perché collega ottimizzazione sotto incertezza, simulazione di processi stocastici discreti e teoria del comportamento animale.

2. Motivazione generale

Il problema del foraging descrive come un animale, un robot autonomo o un agente che cerca risorse distribuisce il proprio tempo tra l'esplorazione di nuove zone e lo sfruttamento delle zone già trovate.

Esempi naturali sono:

- animali che cercano cibo in un paesaggio frammentato;
- api che raccolgono nettare in fiori diversi;
- robot di ricerca e soccorso che ispezionano aree distinte;
- agenti economici che esplorano opportunità di mercato;

- algoritmi di ottimizzazione che bilanciano exploration e exploitation.

In tutti questi casi, la risorsa e' distribuita in patch separate. Ogni patch fornisce cibo a un tasso che decresce nel tempo man mano che la risorsa viene consumata. Quando vale la pena spostarsi verso una patch nuova?

Questa e' la domanda centrale del modello. La risposta, nell'impostazione deterministica classica, e' data dal teorema del valore marginale di Charnov. In una versione stocastica, la domanda diventa piu' ricca: quanto costa l'incertezza? Come si comporta un agente che non conosce esattamente la qualita' delle patch o il tempo necessario per raggiungerle?

3. Struttura dell'ambiente

3.1 Patch e risorse

Supponiamo che l'ambiente sia composto da un numero molto grande di patch, ognuna delle quali contiene una certa quantita' di risorsa. Il forager visita le patch in sequenza.

Ogni patch k e' caratterizzata da:

- un contenuto iniziale di risorsa $Q_k \geq 0$;
- una funzione di guadagno $g(t)$ che descrive quanto cibo e' stato raccolto dopo un tempo t trascorso nella patch.

La funzione $g(t)$ e' tipicamente crescente e concava:

$$g'(t) > 0, \quad g''(t) < 0, \quad \lim_{t \rightarrow \infty} g(t) = Q_k.$$

La concavita' riflette il fatto che la risorsa si esaurisce progressivamente: all'inizio si trova facilmente, poi diventa sempre piu' rara.

3.2 Funzione di guadagno

Una forma funzionale molto usata e' quella esponenzialmente decrescente:

$$g(t) = Q (1 - e^{-\lambda t}),$$

dove:

- Q e' la quantita' totale di risorsa nella patch;
- $\lambda > 0$ e' il tasso di scoperta o di raccolta.

Il tasso istantaneo di guadagno e'

$$g'(t) = Q\lambda e^{-\lambda t},$$

che e' decrescente: il forager raccoglie piu' velocemente all'inizio e piu' lentamente man mano che la patch si depaupera.

3.3 Tempo di viaggio

Per spostarsi da una patch a un'altra, il forager impiega un tempo di viaggio $\tau \geq 0$. Durante il viaggio non raccoglie risorse.

Il tempo di viaggio e' una variabile fondamentale del modello: se il paesaggio e' molto frammentato e le patch sono lontane, il costo di spostarsi e' alto e conviene restare piu' a lungo in ogni patch.

3.4 Tasso di intake

Il tasso di intake energetico medio lungo un percorso che comprende n patch e'

$$\bar{r} = \frac{\text{totale risorsa raccolta}}{\text{totale tempo impiegato}}.$$

Se il forager trascorre un tempo t_k nella patch k e percorre un tempo di viaggio τ_k per raggiungerla, allora

$$\bar{r} = \frac{\sum_{k=1}^n g(t_k)}{\sum_{k=1}^n (t_k + \tau_k)}.$$

L'obiettivo del forager e' massimizzare $\bar{r}$.

4. Il teorema del valore marginale

4.1 Problema deterministico

Nel caso deterministico, tutte le patch sono identiche con contenuto Q e tasso λ , e il tempo di viaggio tra le patch e' costante uguale a τ .

Il forager deve scegliere quanto tempo t^* trascorrere in ogni patch per massimizzare il tasso di intake medio.

Per un'unica patch, il guadagno totale in un ciclo di durata $t + \tau$ e'

$$\bar{r}(t) = \frac{g(t)}{t + \tau}.$$

Si vuole massimizzare questa quantita' rispetto a t .

4.2 Condizione di ottimalita'

Derivando e uguagliando a zero:

$$\frac{d}{dt}\bar{r}(t) = \frac{g'(t)(t + \tau) - g(t)}{(t + \tau)^2} = 0.$$

La condizione di ottimalita' e' quindi

$$g'(t^*) = \frac{g(t^*)}{t^* + \tau} = \bar{r}^*.$$

Questo e' il **teorema del valore marginale**: il forager ottimale lascia la patch quando il tasso istantaneo di guadagno nella patch scende al livello del tasso medio di guadagno nell'ambiente.

4.3 Interpretazione geometrica

Il teorema ha un'interpretazione grafica molto elegante. Se si traccia la curva $g(t)$ a partire dal punto $(-\tau, 0)$, il tempo ottimale t^* corrisponde al punto in cui la tangente alla curva passa per l'origine spostata.

In pratica:

- se τ e' grande, il forager deve aspettare piu' a lungo in ogni patch prima di spostarsi;
- se τ e' piccolo, conviene spostarsi piu' spesso.

Questo risultato formalizza un'intuizione molto naturale: quanto piu' e' costoso spostarsi, tanto piu' conviene sfruttare a fondo la patch corrente.

4.4 Estensione a patch eterogenee

Se le patch hanno contenuti diversi Q_k , la condizione di ottimalita' si generalizza: il forager dovrebbe lasciare ogni patch quando il tasso istantaneo locale raggiunge il tasso medio ambientale $\bar{r}^*$.

Poiche' $\bar{r}^*$ e' globale e dipende dalla composizione dell'ambiente, questa condizione e' intrinsecamente self-consistent: il tasso ottimale di leaving dipende dal tasso medio che a sua volta dipende dal comportamento di leaving.

5. Versione stocastica del modello

5.1 Perche' serve la stocasticita'

Il modello deterministico e' utile come benchmark teorico, ma presenta alcune limitazioni:

- le patch reali hanno qualita' molto variabile;
- il forager spesso non conosce in anticipo la qualita' della patch corrente;
- il tempo di viaggio tra patch dipende dalla geometria casuale del paesaggio;
- il guadagno in ogni visita contiene fluttuazioni stocastiche.

Per questi motivi, una versione piu' realistica introduce la stocasticita' in due luoghi: nella qualita' delle patch e nel tempo di viaggio.

5.2 Patch con qualita' casuale

Supponiamo che il contenuto di ogni patch sia estratto da una distribuzione:

$$Q_k \sim F_Q.$$

Una scelta semplice e' la distribuzione esponenziale:

$$Q_k \sim \text{Exp}(\mu_Q), \quad \mathbb{E}[Q_k] = \mu_Q.$$

In questo caso, il forager arriva in una patch senza sapere quanto cibo contiene. Deve stimare la qualita' della patch in base al guadagno osservato nel tempo.

5.3 Tempo di viaggio casuale

Analogamente, il tempo di viaggio tra patch puo' essere estratto da una distribuzione:

$$\tau_k \sim F_\tau.$$

Una scelta comune e' la distribuzione esponenziale o la distribuzione gamma.

Il tempo di viaggio casuale introduce variabilita' nel costo di spostarsi, rendendo piu' difficile la calibrazione della soglia di leaving.

5.4 Guadagno con rumore

Si puo' aggiungere rumore anche al processo di raccolta. Se in ogni passo temporale il guadagno e' una variabile casuale con media $g'(t)\Delta t$, il guadagno cumulato diventa una traiettoria stocastica:

$$G(t) = g(t) + \sigma W(t),$$

dove $W(t)$ e' un moto browniano o una somma di shock discreti.

Questa componente e' interessante perche' introduce errore di stima nella valutazione della qualita' della patch.

6. Strategie di leaving

Una delle domande principali del progetto e' come il forager decide quando lasciare una patch. Consideriamo quattro strategie.

6.1 Leaving a tempo fisso

Il forager trascorre sempre lo stesso tempo t_{fix} in ogni patch, indipendentemente da quanto ha raccolto.

Questa regola e' semplice e non richiede nessuna valutazione della qualita' della patch. E' ottimale solo se tutte le patch sono identiche e il forager conosce esattamente il tempo ottimale t^* .

6.2 Leaving per soglia di tasso

Il forager osserva il tasso istantaneo di guadagno e lascia la patch quando questo scende sotto una soglia θ :

$$\text{lascia la patch se } g'(t) < \theta.$$

Questa regola avvicina il comportamento ottimale prescritto dal teorema del valore marginale, a patto che il forager riesca a stimare il tasso istantaneo. In pratica, si puo' usare una media mobile del guadagno su una finestra temporale.

6.3 Regola del valore marginale approssimata

Il forager mantiene una stima $\hat{r}$ del tasso di intake medio e lascia la patch quando il tasso corrente scende sotto $\hat{r}$:

$$\text{lascia la patch se } g'(t) < \hat{r}.$$

La stima $\hat{r}$ viene aggiornata dopo ogni visita come media mobile esponenziale:

$$\hat{r} \leftarrow (1 - \alpha)\hat{r} + \alpha \frac{g(t_{\text{visit}})}{t_{\text{visit}} + \tau}.$$

Questo e' il piu' sofisticato dei criteri considerati, perche' adatta la soglia all'esperienza pregressa del forager.

6.4 Leaving casuale

Come baseline, il forager lascia la patch a ogni passo temporale con probabilita' costante p_{leave} :

$$\text{lascia la patch con probabilita' } p_{\text{leave}} \text{ per unita' di tempo.}$$

Questa regola produce tempi di residenza distribuiti esponenzialmente. E' molto semplice da implementare e serve come confronto inferiore.

7. Dinamica temporale di una visita

Una singola visita a una patch con la strategia di leaving per soglia si puo' descrivere come segue.

Il forager arriva nella patch al tempo 0 con guadagno cumulato $G(0) = 0$.

A ogni passo temporale Δt :

1. si raccoglie una quantita' di cibo $\Delta G = g'(t)\Delta t + \text{rumore}$;
2. si aggiorna il guadagno cumulato $G(t + \Delta t) = G(t) + \Delta G$;
3. si stima il tasso corrente $\hat{g}'(t)$;
4. se $\hat{g}'(t) < \theta$, il forager lascia la patch;
5. altrimenti si continua.

Il tempo di leaving T_{leave} e' una variabile casuale la cui distribuzione dipende dalla qualita' della patch, dalla funzione di guadagno e dalla soglia θ .

8. Tasso di intake medio e confronto tra strategie

Dati S cicli di foraging indipendenti, il tasso di intake medio empirico e'

$$\hat{r} = \frac{\sum_{s=1}^S G_s(T_s)}{\sum_{s=1}^S (T_s + \tau_s)},$$

dove $G_s(T_s)$ e' il guadagno nella visita s , T_s il tempo di residenza e τ_s il tempo di viaggio.

Questa osservabile e' la principale metrica di confronto tra strategie.

8.1 Osservabili secondarie

Oltre al tasso di intake, e' utile misurare:

- la distribuzione dei tempi di residenza T_s ;
- la distribuzione dei guadagni per visita G_s ;
- la media e la varianza del guadagno cumulato su un orizzonte temporale T_{tot} ;
- la frequenza con cui il forager abbandona patch di alta qualita' troppo presto o rimane in patch povere troppo a lungo.

8.2 Effetto della variabilita' ambientale

Una delle domande piu' interessanti del progetto e' come cambia il tasso ottimale al variare della variabilita' delle patch.

Se le patch sono molto eterogenee, la strategia ottimale deve adattarsi rapidamente alle differenze. Una soglia fissa puo' essere troppo alta per le patch povere (il forager le abbandona troppo presto) o troppo bassa per le patch ricche (il forager resta troppo a lungo).

Questo introduce un compromesso tra semplicita' della regola e capacita' di adattamento.

9. Connessione con il random walk

9.1 Foraging come random walk

Una prospettiva alternativa consiste nel leggere il foraging come un problema di random walk in un paesaggio di risorse.

Se i candidati alla patch sono distribuiti casualmente nel piano, il forager esegue un random walk bidimensionale alla ricerca di nuove patch. La distanza percorsa tra una patch e la successiva determina il tempo di viaggio.

In questo schema, la distribuzione dei tempi di viaggio non e' piu' esogena ma emerge dalla geometria del paesaggio.

9.2 Levy flight e ricerca ottimale

Un risultato molto citato nella letteratura e' che, in un paesaggio sparso con patch rare, un random walk con spostamenti distribuiti secondo una legge di potenza (Levy flight) puo' essere piu' efficiente di un random walk browniano standard.

L'intuizione e' che spostamenti occasionalmente molto lunghi permettono di esplorare regioni dell'ambiente non ancora visitate, riducendo la probabilita' di ritornare piu' volte nelle stesse zone.

Dal punto di vista del corso, questo collegamento e' molto utile perche' introduce la distribuzione a code pesanti in un contesto applicativo molto concreto.

9.3 Confronto tra random walk browniano e Levy flight

Una versione semplice del confronto consiste nel:

1. generare un paesaggio con patch distribuite casualmente nel piano;
2. simulare un forager con spostamenti gaussiani (random walk browniano);
3. simulare un forager con spostamenti distribuiti secondo una Pareto troncata (Levy flight);
4. confrontare il numero medio di patch visitate in un tempo fissato;

5. confrontare il tasso di intake per i due tipi di movimento.

Questo esperimento e' computazionalmente semplice ma concettualmente molto ricco.

10. Domande scientifiche che il modello permette di studiare

Il modello e' utile per affrontare domande precise.

1. Quanto si avvicina la regola del valore marginale approssimata alla soluzione ottimale deterministica?
2. Come cambia il tempo ottimale di residenza al variare del tempo di viaggio medio?
3. La variabilita' nella qualita' delle patch riduce o aumenta il tasso di intake rispetto al caso omogeneo?
4. Qual e' l'errore commesso da una strategia a tempo fisso calibrata sul caso deterministico?
5. Il Levy flight produce un tasso di intake superiore al random walk browniano in tutti i tipi di paesaggio?
6. Come si comporta la regola adattiva del valore marginale approssimata quando l'ambiente cambia nel tempo?

11. Pseudocodice del modello base

11.1 Input

- funzione di guadagno $g(t)$ e suo tasso $g'(t)$
- distribuzione della qualita' delle patch F_Q
- distribuzione del tempo di viaggio F_τ
- strategia di leaving scelta
- parametri della strategia (soglia θ , tempo fisso t_{fix} , ecc.)
- numero di cicli S
- passo temporale Δt

11.2 Pseudocodice

1. inizializza $\hat{r} = 0$, totale_guadagno = 0, totale_tempo = 0
2. per $s = 1, \dots, S$:
 - estrai il tempo di viaggio $\tau_s \sim F_\tau$
 - estrai la qualita' della patch $Q_s \sim F_Q$
 - aggiorna totale_tempo $+= \tau_s$
 - inizializza $t = 0$, $G = 0$
 - ripeti fino al leaving:
 - calcola il guadagno istantaneo $\Delta G = g'(t; Q_s)\Delta t$

- aggiorna $G \leftarrow G + \Delta G$, $t \leftarrow t + \Delta t$
- applica la regola di leaving
 - aggiorna $\text{totale_guadagno} \leftarrow \text{totale_guadagno} + G$, $\text{totale_tempo} \leftarrow \text{totale_tempo} + t$
 - aggiorna $\hat{r}$ se si usa la regola adattiva
- 3. calcola $\hat{r}_{\text{finale}} = \text{totale_guadagno} / \text{totale_tempo}$
- 4. salva le distribuzioni di T_s e G_s

11.3 Nota sul passo temporale

La discretizzazione in passi Δt e' sufficiente per una prima implementazione. Per la strategia a soglia di tasso, e' importante che Δt sia abbastanza piccolo da non saltare la soglia; per la strategia adattiva, e' utile aggiornare $\hat{r}$ solo dopo ogni visita completa, non ad ogni passo.

12. Schema del laboratorio

12.1 Laboratorio 1 - Soluzione deterministica e teorema del valore marginale

Obiettivo

Verificare numericamente il teorema del valore marginale e comprendere come il tempo ottimale dipende dal tempo di viaggio.

Attività

1. fissare la funzione $g(t) = Q(1 - e^{-\lambda t})$ con Q e λ dati;
2. calcolare numericamente t^* come massimo di $g(t)/(t + \tau)$ per diversi valori di τ ;
3. confrontare il risultato con la condizione $g'(t^*) = g(t^*)/(t^* + \tau)$;
4. costruire il grafico di t^* in funzione di τ .

Domande guida

- t^* cresce o decresce con τ ?
- come cambia il tasso ottimale $\bar{r}^*$ al variare di τ ?
- il tasso ottimale e' una funzione monotona di Q ?

Output richiesto

- codice sorgente;
- grafici di $g(t)/(t + \tau)$ e della condizione di ottimalità;
- tabella di t^* e $\bar{r}^*$ per diversi valori di τ .

12.2 Laboratorio 2 - Confronto tra strategie di leaving

Obiettivo

Simulare foraging con patch omogenee e confrontare le quattro strategie.

Attività

1. fissare Q , λ , τ costanti;
2. implementare le quattro strategie;
3. simulare $S = 1000$ cicli per ogni strategia;
4. confrontare $\hat{r}$ e le distribuzioni di T_s .

Domande guida

- quale strategia si avvicina di più al tasso ottimale deterministico?
- la strategia a tempo fisso calibrata su t^* è competitiva?
- quanto conta la varianza di T_s nel determinare il tasso medio?

Output richiesto

- tabella dei tassi di intake per le diverse strategie;
- istogrammi dei tempi di residenza;
- commento interpretativo.

12.3 Laboratorio 3 - Patch eterogenee e variabilità ambientale

Obiettivo

Studiare come la variabilità nella qualità delle patch modifica il tasso di intake e il comportamento ottimale.

Attività

1. introdurre patch con $Q_k \sim \text{Exp}(\mu_Q)$;
2. ripetere il confronto tra strategie;
3. variare la deviazione standard di Q_k mantenendo la media costante;
4. misurare il tasso di intake medio in funzione della variabilità.

Domande guida

- la variabilità nelle patch avvantaggia o svantaggia il forager?
- la regola adattiva performa meglio delle altre in ambienti più eterogenei?
- esiste un livello di variabilità oltre il quale nessuna strategia semplice è adeguata?

Output richiesto

- grafici del tasso di intake in funzione della deviazione standard di Q ;
- confronto tra strategie al variare dell'eterogeneità;
- discussione qualitativa.

12.4 Laboratorio 4 - Random walk e Levy flight

Obiettivo

Confrontare l'efficienza di esplorazione tra random walk browniano e Levy flight in un paesaggio con patch sparse.

Attività

1. generare un paesaggio 2D con P patch distribuite uniformemente in un quadrato;
2. implementare un forager con spostamenti gaussiani;
3. implementare un forager con spostamenti distribuiti secondo una Pareto troncata;
4. confrontare il numero di patch visitate e il guadagno cumulato in un tempo totale fissato.

Domande guida

- in quale regime di densità delle patch il Levy flight è più vantaggioso?
- come cambia il risultato al variare dell'esponente della distribuzione di Pareto?
- la strategia di leaving influenza molto il confronto?

Output richiesto

- mappe delle traiettorie per i due tipi di forager;
- grafici del guadagno cumulato nel tempo;
- commento sui regimi di densità.

13. Una possibile estensione teorica

13.1 Foraging e processi di rinforzo

Una estensione molto naturale consiste nel trattare il forager come un agente che apprende le proprietà dell'ambiente attraverso l'esperienza.

Se dopo ogni visita il forager aggiorna la propria stima della qualità media delle patch usando una media mobile esponenziale, il sistema diventa un processo adattivo di rinforzo positivo: le patch che hanno reso molto in passato tendono ad essere visitate più spesso.

Questo schema connette il foraging al piu' generale problema dell'exploration-exploitation, che compare in molti contesti del corso: modello di March, esternalita' di rete, dinamiche replicative.

13.2 Foraging su rete

Se le patch non sono distribuite nel piano ma sono i nodi di un grafo, il problema del foraging si trasforma in un problema di random walk su rete. Il tempo di viaggio tra due patch dipende dalla distanza sul grafo.

In questo schema si possono usare reti con strutture diverse (random, scale-free, reticolare) e studiare come la topologia influenzi il tasso di intake.

14. Perche' questo e' un buon case study per il corso

Questo progetto e' particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, introduce una forma di ottimizzazione sotto incertezza molto concreta e visivamente intuitiva.

Secondo, il modello deterministico e' semplice da analizzare analiticamente, il che permette di avere un benchmark teorico preciso con cui confrontare i risultati della simulazione.

Terzo, la versione stocastica introduce in modo molto naturale:

- variabili casuali con distribuzioni diverse;
- simulazione Monte Carlo;
- confronto tra strategie;
- stima empirica di osservabili.

Quarto, il collegamento con il random walk e il Levy flight permette di introdurre distribuzioni a code pesanti in un contesto applicativo molto motivante.

Quinto, il tema dell'exploration-exploitation connette il progetto ad altri modelli del corso, creando una coerenza tematica trasversale.

15. Conclusione

Il problema del foraging ottimale mostra come una domanda semplice — quanto tempo restare in una patch? — generi una teoria matematica ricca e un programma di simulazione molto naturale.

Il teorema del valore marginale fornisce la risposta deterministica. Le versioni stocastiche mostrano che l'incertezza sulla qualita' delle patch, i tempi di

viaggio variabili e il rumore nella raccolta modificano in modo non banale il comportamento ottimale.

Dal punto di vista metodologico, questo progetto combina in modo molto naturale:

- ottimizzazione continua;
- simulazione stocastica discreta;
- confronto tra strategie;
- distribuzioni empiriche;
- collegamento con random walk e processi adattativi.

Il messaggio concettuale piu' importante e' che l'ottimale teorico non e' sempre raggiungibile in pratica, e che regole semplici e adattive possono avvicinarsi molto alla soluzione ottimale con un costo computazionale molto basso.

16. Bibliografia minima

1. Charnov, E. L. (1976). Optimal Foraging, the Marginal Value Theorem. *Theoretical Population Biology*, 9(2), 129-136.
2. Stephens, D. W., and Krebs, J. R. (1986). *Foraging Theory*. Princeton University Press.
3. Viswanathan, G. M., Buldyrev, S. V., Havlin, S., da Luz, M. G. E., Raposo, E. P., and Stanley, H. E. (1999). Optimizing the Success of Random Searches. *Nature*, 401, 911-914.
4. Pyke, G. H. (1984). Optimal Foraging Theory: A Critical Review. *Annual Review of Ecology and Systematics*, 15, 523-575.
5. Hills, T. T., Todd, P. M., Lazer, D., Redish, A. D., and Couzin, I. D. (2015). Exploration versus Exploitation in Space, Mind, and Society. *Trends in Cognitive Sciences*, 19(1), 46-54.

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python il modello di foraging ottimale.

L'obiettivo non e' costruire un simulatore sofisticato, ma fornire una guida leggibile che possa essere letta:

- come pseudocodice da chi usa altri linguaggi;
- come base quasi immediatamente eseguibile da chi conosce Python.

Per questo motivo il codice e' volutamente elementare:

- poche librerie;

- funzioni corte;
- cicli ed espressioni esplicite;
- nomi leggibili.

La logica generale e':

1. definire la funzione di guadagno e il suo tasso;
2. implementare le quattro strategie di leaving;
3. simulare una singola visita a una patch;
4. simulare una sequenza di visite (un percorso di foraging);
5. confrontare le strategie su molte simulazioni;
6. implementare il confronto tra random walk browniano e Levy flight.

A.1 Librerie minime

```
import random
import math
import statistics
import matplotlib.pyplot as plt
```

Quindi:

- `random` serve per estrarre variabili casuali;
- `math` serve per funzioni elementari;
- `statistics` serve per medie e deviazioni standard;
- `matplotlib.pyplot` serve per i grafici.

Non e' necessario usare `numpy` in una prima implementazione.

A.2 Funzione di guadagno

La funzione di guadagno standard e' quella esponenzialmente decrescente:

$$g(t; Q, \lambda) = Q(1 - e^{-\lambda t}).$$

```
def gain(t, Q, lam):
    return Q * (1.0 - math.exp(-lam * t))
```

Il tasso istantaneo di guadagno e':

$$g'(t; Q, \lambda) = Q\lambda e^{-\lambda t}.$$

```
def gain_rate(t, Q, lam):
    return Q * lam * math.exp(-lam * t)
```

Queste due funzioni sono il cuore del modello.

A.3 Soluzione deterministica del teorema del valore marginale

Per trovare t^* numericamente si può cercare il massimo di $g(t)/(t + \tau)$:

```
def optimal_residence_time(Q, lam, tau, t_max=20.0, num_points=2000):
    best_t = 0.0
    best_rate = 0.0

    for n in range(1, num_points + 1):
        t = t_max * n / num_points
        rate = gain(t, Q, lam) / (t + tau)

        if rate > best_rate:
            best_rate = rate
            best_t = t

    return best_t, best_rate
```

Esempio:

```
t_star, r_star = optimal_residence_time(Q=10.0, lam=0.5, tau=2.0)
print("t* =", t_star)
print("r* =", r_star)
```

A.4 Estrazione delle variabili casuali

Qualità della patch

```
def sample_patch_quality(mu_Q):
    return random.expovariate(1.0 / mu_Q)
```

Qui `random.expovariate(rate)` genera una variabile esponenziale con media $1/\text{rate}$, quindi si usa `1.0 / mu_Q` per ottenere media `mu_Q`.

Tempo di viaggio

```
def sample_travel_time(mu_tau):
    return random.expovariate(1.0 / mu_tau)
```

Per un tempo di viaggio costante si usa semplicemente il valore fisso `tau` senza campionamento.

A.5 Strategie di leaving

Ogni strategia viene implementata come una funzione che, dati lo stato corrente della visita, decide se lasciare la patch.

Strategia 1: tempo fisso

```
def leaving_fixed_time(t, G, t_fix, **kwargs):  
    return t >= t_fix
```

Strategia 2: soglia di tasso

```
def leaving_rate_threshold(t, G, Q, lam, threshold, **kwargs):  
    current_rate = gain_rate(t, Q, lam)  
    return current_rate < threshold
```

Nota: in questa versione il forager conosce i parametri della patch. Una versione piu' realistica stima il tasso dalla storia del guadagno osservato.

Strategia 3: valore marginale approssimato

```
def leaving_marginal_value(t, G, tau, r_hat, **kwargs):  
    if t <= 0.0:  
        return False  
    current_rate = G / t  
    return current_rate < r_hat
```

Qui r_hat e' la stima corrente del tasso medio ambientale, aggiornata dopo ogni visita.

Strategia 4: leaving casuale

```
def leaving_random(t, G, p_leave, dt, **kwargs):  
    return random.random() < p_leave * dt
```

A.6 Simulazione di una singola visita

```
def simulate_visit(Q, lam, strategy, strategy_params, dt=0.01, t_max=50.0):  
    t = 0.0  
    G = 0.0  
  
    while t < t_max:  
        dG = gain_rate(t, Q, lam) * dt  
        G += dG  
        t += dt  
  
        leave = strategy(  
            t=t,  
            G=G,  
            Q=Q,  
            lam=lam,  
            **strategy_params  
        )
```

```

        if leave:
            break

    return t, G

```

Qui `strategy` e' una funzione di leaving e `strategy_params` e' un dizionario dei suoi parametri.

Esempio con soglia di tasso:

```

t_visit, G_visit = simulate_visit(
    Q=10.0,
    lam=0.5,
    strategy=leaving_rate_threshold,
    strategy_params={"threshold": 0.8},
    dt=0.01
)
print("Tempo di residenza:", t_visit)
print("Guadagno:", G_visit)

```

A.7 Simulazione di un percorso di foraging completo

```

def simulate_foraging(num_cycles, mu_Q, mu_tau, strategy, strategy_params,
                     dt=0.01, t_max=50.0, adaptive_rate=False, alpha=0.1):
    total_gain = 0.0
    total_time = 0.0

    history_t = []
    history_G = []
    history_r = []

    r_hat = 0.0

    for cycle in range(num_cycles):
        tau = sample_travel_time(mu_tau)
        Q = sample_patch_quality(mu_Q)

        total_time += tau

        if adaptive_rate:
            strategy_params["r_hat"] = r_hat

    t_visit, G_visit = simulate_visit(
        Q=Q,
        lam=strategy_params.get("lam", 0.5),
        strategy=strategy,

```

```

        strategy_params=strategy_params,
        dt=dt,
        t_max=t_max
    )

    total_gain += G_visit
    total_time += t_visit

    if total_time > 0.0:
        r_current = total_gain / total_time
    else:
        r_current = 0.0

    if adaptive_rate and cycle > 0:
        cycle_rate = G_visit / (t_visit + tau)
        r_hat = (1.0 - alpha) * r_hat + alpha * cycle_rate

    history_t.append(t_visit)
    history_G.append(G_visit)
    history_r.append(r_current)

mean_intake_rate = total_gain / total_time if total_time > 0.0 else 0.0

results = {
    "mean_intake_rate": mean_intake_rate,
    "history_residence_times": history_t,
    "history_gains": history_G,
    "history_cumulative_rate": history_r
}

return results

```

A.8 Confronto tra strategie

Per confrontare le quattro strategie in modo sistematico:

```

def compare_strategies(num_cycles, mu_Q, mu_tau, lam,
                       t_fix, threshold, p_leave, alpha_adapt,
                       dt=0.01, t_max=50.0):

    results = {}

    # Strategia 1: tempo fisso
    results["fixed_time"] = simulate_foraging(
        num_cycles=num_cycles,
        mu_Q=mu_Q,

```

```

        mu_tau=mu_tau,
        strategy=leaving_fixed_time,
        strategy_params={"t_fix": t_fix, "lam": lam},
        dt=dt,
        t_max=t_max
    )

    # Strategia 2: soglia di tasso
    results["rate_threshold"] = simulate_foraging(
        num_cycles=num_cycles,
        mu_Q=mu_Q,
        mu_tau=mu_tau,
        strategy=leaving_rate_threshold,
        strategy_params={"threshold": threshold, "lam": lam},
        dt=dt,
        t_max=t_max
    )

    # Strategia 3: valore marginale approssimato
    results["marginal_value"] = simulate_foraging(
        num_cycles=num_cycles,
        mu_Q=mu_Q,
        mu_tau=mu_tau,
        strategy=leaving_marginal_value,
        strategy_params={"tau": mu_tau, "r_hat": 0.0, "lam": lam},
        dt=dt,
        t_max=t_max,
        adaptive_rate=True,
        alpha=alpha_adapt
    )

    # Strategia 4: leaving casuale
    results["random_leaving"] = simulate_foraging(
        num_cycles=num_cycles,
        mu_Q=mu_Q,
        mu_tau=mu_tau,
        strategy=leaving_random,
        strategy_params={"p_leave": p_leave, "dt": dt, "lam": lam},
        dt=dt,
        t_max=t_max
    )

    return results

```

A.9 Grafici delle traiettorie

Distribuzione dei tempi di residenza

```
def plot_residence_times(results_dict, bins=40):
    for name, results in results_dict.items():
        plt.hist(
            results["history_residence_times"],
            bins=bins,
            alpha=0.5,
            density=True,
            label=name
        )

    plt.xlabel("tempo di residenza")
    plt.ylabel("densita' empirica")
    plt.title("Distribuzione dei tempi di residenza")
    plt.legend()
    plt.show()
```

Tasso di intake cumulato nel tempo

```
def plot_cumulative_rate(results_dict):
    for name, results in results_dict.items():
        cycles = list(range(len(results["history_cumulative_rate"])))
        plt.plot(cycles, results["history_cumulative_rate"], label=name)

    plt.xlabel("ciclo")
    plt.ylabel("tasso di intake cumulato")
    plt.title("Convergenza del tasso di intake")
    plt.legend()
    plt.show()
```

Tabella riassuntiva

```
def print_summary(results_dict, r_star):
    print("Strategia          | tasso di intake | scarto da ottimo")
    print("-" * 60)

    for name, results in results_dict.items():
        r = results["mean_intake_rate"]
        gap = r_star - r
        print(f"{name:<25} | {r:.4f}          | {gap:.4f}")
```

Esempio completo:

```
t_star, r_star = optimal_residence_time(Q=10.0, lam=0.5, tau=2.0)
```

```

results = compare_strategies(
    num_cycles=2000,
    mu_Q=10.0,
    mu_tau=2.0,
    lam=0.5,
    t_fix=t_star,
    threshold=r_star,
    p_leave=0.1,
    alpha_adapt=0.05,
    dt=0.01
)

print_summary(results, r_star)
plot_residence_times(results)
plot_cumulative_rate(results)

```

A.10 Confronto al variare del tempo di viaggio

Una delle analisi piu' informative e' la comparativa statica rispetto a τ :

```

def tau_sweep(tau_values, num_cycles, mu_Q, lam,
              alpha_adapt=0.05, dt=0.01, t_max=50.0):
    rates_fixed = []
    rates_threshold = []
    rates_adapt = []
    rates_optimal = []

    for tau in tau_values:
        t_star, r_star = optimal_residence_time(Q=mu_Q, lam=lam, tau=tau)

        res = compare_strategies(
            num_cycles=num_cycles,
            mu_Q=mu_Q,
            mu_tau=tau,
            lam=lam,
            t_fix=t_star,
            threshold=r_star * 0.8,
            p_leave=0.05,
            alpha_adapt=alpha_adapt,
            dt=dt,
            t_max=t_max
        )

        rates_fixed.append(res["fixed_time"]["mean_intake_rate"])
        rates_threshold.append(res["rate_threshold"]["mean_intake_rate"])
        rates_adapt.append(res["marginal_value"]["mean_intake_rate"])

```

```

        rates_optimal.append(r_star)

    return rates_fixed, rates_threshold, rates_adapt, rates_optimal

def plot_tau_sweep(tau_values, rates_fixed, rates_threshold, rates_adapt, rates_optimal):
    plt.plot(tau_values, rates_optimal, label="ottimo teorico", linestyle="--")
    plt.plot(tau_values, rates_fixed, label="tempo fisso")
    plt.plot(tau_values, rates_threshold, label="soglia di tasso")
    plt.plot(tau_values, rates_adapt, label="valore marginale adattivo")
    plt.xlabel("tempo di viaggio medio")
    plt.ylabel("tasso di intake")
    plt.title("Confronto tra strategie al variare del tempo di viaggio")
    plt.legend()
    plt.show()

```

A.11 Random walk e Levy flight nel paesaggio 2D

Generazione del paesaggio

```

def generate_landscape(num_patches, L, mu_Q):
    patches = []

    for _ in range(num_patches):
        x = random.uniform(0.0, L)
        y = random.uniform(0.0, L)
        Q = sample_patch_quality(mu_Q)
        patches.append((x, y, Q))

    return patches

```

Distanza euclidea tra due punti

```

def euclidean_distance(x1, y1, x2, y2):
    return math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

```

Patch piu' vicina non ancora visitata

```

def nearest_unvisited_patch(x, y, patches, visited):
    best_idx = None
    best_dist = float("inf")

    for idx, (px, py, Q) in enumerate(patches):
        if idx in visited:
            continue
        d = euclidean_distance(x, y, px, py)
        if d < best_dist:

```

```

        best_dist = d
        best_idx = idx

    return best_idx, best_dist

```

Spostamento gaussiano (random walk browniano)

```

def gaussian_step(sigma_step):
    dx = random.gauss(0.0, sigma_step)
    dy = random.gauss(0.0, sigma_step)
    return dx, dy

```

Spostamento di Levy (Pareto troncato)

La distribuzione di Pareto con esponente μ ha densita' $p(\ell) \propto \ell^{-\mu}$ per $\ell \geq \ell_{\min}$.
 Si puo' campionare per inversione:

```

def levy_step(mu_levy, l_min=0.1, l_max=100.0):
    u = random.random()
    l = l_min * (1.0 - u * (1.0 - (l_min / l_max) ** (mu_levy - 1.0))) ** (-1.0 / (mu_levy - 1.0))
    angle = random.uniform(0.0, 2.0 * math.pi)
    dx = l * math.cos(angle)
    dy = l * math.sin(angle)
    return dx, dy

```

Simulazione del forager su paesaggio 2D

```

def simulate_forager_2d(patches, L, lam, t_fix,
                        total_time_budget, dt_move, movement="brownian",
                        sigma_step=1.0, mu_levy=2.0):
    x, y = random.uniform(0.0, L), random.uniform(0.0, L)
    visited = set()
    total_gain = 0.0
    time_elapsed = 0.0

    trajectory_x = [x]
    trajectory_y = [y]

    while time_elapsed < total_time_budget:
        # cerca la patch piu' vicina non visitata
        idx, dist = nearest_unvisited_patch(x, y, patches, visited)

        if idx is None:
            break

        # tempo di viaggio proporzionale alla distanza
        travel_time = dist

```

```

time_elapsed += travel_time

if time_elapsed >= total_time_budget:
    break

# raccolta nella patch
px, py, Q = patches[idx]
x, y = px, py
visited.add(idx)

t_visit = min(t_fix, total_time_budget - time_elapsed)
G = gain(t_visit, Q, lam)

total_gain += G
time_elapsed += t_visit

trajectory_x.append(x)
trajectory_y.append(y)

results = {
    "total_gain": total_gain,
    "patches_visited": len(visited),
    "trajectory_x": trajectory_x,
    "trajectory_y": trajectory_y
}

return results

```

Nota: in questa versione semplificata il forager si sposta direttamente verso la patch piu' vicina non visitata. Una versione piu' realistica userebbe spostamenti casuali per esplorare il paesaggio.

Confronto browniano vs Levy

```

def compare_movement_strategies(num_landscapes, num_patches, L, mu_Q, lam,
                                t_fix, total_time_budget,
                                sigma_step=1.0, mu_levy=2.0):

    gains_brownian = []
    patches_brownian = []
    gains_levy = []
    patches_levy = []

    for trial in range(num_landscapes):
        patches = generate_landscape(num_patches, L, mu_Q)

        res_b = simulate_forager_2d(

```

```

        patches=patches,
        L=L,
        lam=lam,
        t_fix=t_fix,
        total_time_budget=total_time_budget,
        dt_move=0.1,
        movement="brownian",
        sigma_step=sigma_step
    )

    res_l = simulate_forager_2d(
        patches=patches,
        L=L,
        lam=lam,
        t_fix=t_fix,
        total_time_budget=total_time_budget,
        dt_move=0.1,
        movement="levy",
        mu_levy=mu_levy
    )

    gains_brownian.append(res_b["total_gain"])
    patches_brownian.append(res_b["patches_visited"])
    gains_levy.append(res_l["total_gain"])
    patches_levy.append(res_l["patches_visited"])

    summary = {
        "mean_gain_brownian": statistics.mean(gains_brownian),
        "mean_gain_levy": statistics.mean(gains_levy),
        "mean_patches_brownian": statistics.mean(patches_brownian),
        "mean_patches_levy": statistics.mean(patches_levy)
    }

    return summary

```

A.12 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo in questo ordine:

1. import delle librerie;
2. funzioni di base:
 - gain
 - gain_rate
 - sample_patch_quality
 - sample_travel_time
3. soluzione deterministica:

- optimal_residence_time
- 4. strategie di leaving:
 - leaving_fixed_time
 - leaving_rate_threshold
 - leaving_marginal_value
 - leaving_random
- 5. simulazione:
 - simulate_visit
 - simulate_foraging
 - compare_strategies
- 6. grafici:
 - plot_residence_times
 - plot_cumulative_rate
 - plot_tau_sweep
- 7. paesaggio 2D:
 - generate_landscape
 - simulate_forager_2d
 - compare_movement_strategies
- 8. blocco finale con esempi.

Per esempio:

```
if __name__ == "__main__":
    Q = 10.0
    lam = 0.5
    tau = 2.0

    t_star, r_star = optimal_residence_time(Q=Q, lam=lam, tau=tau)
    print("Soluzione deterministica:")
    print("  t* =", round(t_star, 3))
    print("  r* =", round(r_star, 4))

    results = compare_strategies(
        num_cycles=1000,
        mu_Q=Q,
        mu_tau=tau,
        lam=lam,
        t_fix=t_star,
        threshold=r_star,
        p_leave=0.1,
        alpha_adapt=0.05,
        dt=0.01
    )

    print_summary(results, r_star)
    plot_residence_times(results)
    plot_cumulative_rate(results)
```

A.13 Perché questa appendice è utile

Questa appendice ha due funzioni didattiche.

Primo, mostra che il passaggio da un modello teorico a una simulazione è molto diretto: la funzione di guadagno e le regole di leaving si traducono quasi letteralmente in codice.

Secondo, la struttura modulare permette di confrontare quattro strategie semplicemente cambiando un parametro nella chiamata a `compare_strategies`, rendendo molto semplice l'analisi parametrica.

A.14 Conclusione dell'appendice

La struttura proposta è volutamente semplice. Chi conosce Python può implementarla quasi direttamente; chi usa altri linguaggi può leggerla come pseudocodice molto vicino a una traduzione operativa.

Il messaggio metodologico è che anche un modello di ottimizzazione classico come il foraging diventa molto più ricco non appena si introduce la stocasticità: le patch hanno qualità variabile, i tempi di viaggio fluttuano, e il forager deve adattarsi senza conoscere in anticipo la struttura dell'ambiente.

È proprio questo passaggio dal determinismo all'incertezza che rende il foraging un caso di studio così utile per un corso di metodi computazionali per modelli stocastici.